

SIMT-Aware Lockstep Verification and Functional-Coverage Closure Methodology for an Open-Source RISC-V GPGPU: A Complete UVM 1.2 Environment

Samuel Moussa^{a,1}, Steven Ibrahim^a, Ahmad Sudky^a, Ahmad Fawzy^a,
Abanoub Nabil^a, Alhassan Sayed^a, Hossam Hassan^b, Hyung-Min Yoon^c

^a*Department of Electronics and Communications Engineering, Faculty of
Engineering, Minia, Egypt*

^b*Independent Researcher, South Korea*

^c*Siliconarts, Inc., South Korea*

Abstract

Open-source RISC-V GPGPUs ship without reference-model verification, functional-coverage models, or sign-off discipline, while concurrent fuzzing work demonstrates that real defects populate exactly this design class. This paper presents a complete UVM 1.2 verification environment and closure methodology for the Vortex RISC-V GPGPU, and the findings it produced on both sides of the comparison. The environment wraps a bus-master SIMT device with role-inverted agents, integrates the project's functional simulator as a per-configuration golden model over DPI-C, and renders verdicts through two checkers that are themselves qualified by permanent fault-injection guards: a bidirectional end-state scoreboard and a per-instruction, per-lane lockstep comparator governed by five alignment rules that we semi-

*Corresponding author.

Email address: samuelmoussa64@gmail.com (Samuel Moussa)

formalize as precondition–soundness properties. A two-pass load-value feed makes fenceless multi-core programs instruction-granularity verifiable—verified modulo the fed race resolutions, with the assumption stated and, in the flagship case, discharged by a trace-level race witness. A three-layer coverage model adds what is, to our knowledge, the first published SIMT functional-coverage layer for RTL GPU verification (IPDOM divergence depth, scratch-pad bank conflicts, coalescing classes), closed under machine-generated, RTL-cited exclusions enforced by a blocking waiver-integrity gate. The methodology surfaced externally corroborated RTL defects (a JALR LSB ISA deviation, a reset-relay X window behind a silenced assertion) and reference-model defects found by the lockstep itself. The checking stack is cheap to operate: lockstep adds no measurable wall-clock overhead at either end of the kernel-size spectrum and roughly 30% peak memory at scale. Stimulus *kind*, not volume, moves the coverage model: ninety additional scalar-generator programs closed zero bins, while fifty-seven axis-directed programs closed the SIMT targets.

Keywords: UVM, RISC-V, GPGPU, SIMT, functional verification, lockstep co-simulation, functional coverage, open-source hardware

1. Introduction

Open-source silicon has an industrial verification problem with a sharpening edge. The RISC-V ecosystem maintains mature verification assets for scalar cores—step-and-compare environments, constrained-random generators, ISA coverage libraries [5, 7, 6, 9, 4, 10]—but its most complex open processor class, the GPGPU, has had none of this. Vortex [1, 2], the leading

open RISC-V GPGPU, ships a directed-kernel regression with no RTL-level reference checking and no functional-coverage model. Concurrently, FuzzGPU [3] demonstrated that a fuzzer can find real defects in exactly this class of design (20 previously unknown bugs, 10 CVEs, across Vortex and Ventus), converting the gap from an academic inconvenience into a security exposure.

This paper closes the other half of that problem: not bug discovery, but *coverage-quantified sign-off*. Its contributions:

1. a complete, configuration-parametric UVM 1.2 [11] environment for a bus-master SIMT GPGPU (Section 3);
2. per-instruction, per-lane lockstep for SIMT via five alignment rules, *semi-formalized* as precondition–soundness properties (Section 5);
3. a two-pass load-value feed that makes fenceless multi-core programs instruction-granularity verifiable, with its soundness assumption stated and, where possible, discharged by trace-level race witnesses (Section 6);
4. a three-layer coverage model including the first published SIMT functional-coverage layer for RTL GPU verification (Section 8);
5. an exclusion methodology—machine-generated, RTL-cited waivers under a blocking hits-invariant merge gate—elevated here to a named, reusable contribution (Section 9);
6. an evidence-classified findings register covering both the RTL and the reference model, with dispositions and external corroboration (Sections 10–10.2), and the verification-cost and per-generator marginal-coverage evidence that adoption requires (Section 11).

All honesty constraints of the conference version carry over verbatim (Section 12); every number in this paper traces to a banked artifact or is marked as pending.

2. Background and related work

2.1. The DUT and its reference models

Vortex organizes as clusters $\rightarrow$ sockets $\rightarrow$ cores $\rightarrow$ warps $\rightarrow$ threads with per-core caches, optional shared L2/L3, an immediate-post-dominator (IPDOM) reconvergence stack, and hardware barriers; the verified build is RV32IMAF at a pinned revision on QuestaSim. Vortex has no trap architecture—a property with verification consequences (Section 10). SimX, its functional simulator, is structurally an RVVI-shaped golden model [6]; Spike [8] cannot execute the SIMT instructions and serves only as a scalar, independent base-ISA cross-check (11,076-retirement three-way audit: zero mismatches, injection-proven non-vacuity). *The SIMT axis has no independent reference*—a structural ceiling we return to in the threats section.

2.2. Reference-model verification of processors

Step-and-compare is the standard of care for RISC-V CPUs. core-verif [5] established the open-source pattern with ImperasDV [9] over RVVI [6]; riscv-dv [7] supplies constrained-random programs with a trace-compare flow; DiffTest [4] industrialized per-instruction architectural comparison over DPI-C for XiangShan; OpenTitan [10] demonstrates sign-off-grade UVM on open silicon at SoC scale. All target scalar or SIMD-vector cores. This work extends the family along three axes: a bus-master DUT with role-inverted

agents, SIMT retirements (per-lane masked compare, split records, divergence), and weakly-ordered multi-core programs (the two-pass feed). Per-warp differential testing on this very DUT was concurrently demonstrated by FuzzGPU; our lockstep differs in the formalized alignment rules, the UVM packaging, and its role as one qualified checker inside a sign-off flow rather than a fuzzer’s oracle.

2.3. GPU kernel verification and testing

Software-level kernel verification is mature: GPUVerify [15] proves race- and divergence-freedom of OpenCL/CUDA kernels; GKLEE [16] performs concolic kernel testing, explicitly targeting divergent warps, non-coalesced accesses, and shared-memory bank conflicts—the same three phenomena our coverage layer quantifies at the RTL level; WEFT [17] verifies producer–consumer synchronization in GPU programs. Learned program generation has precedent in compiler fuzzing [18]. These operate on kernel source, not RTL; our contribution begins where their remit ends.

2.4. RTL GPU testing and memory consistency

FuzzGPU [3] (USENIX Security 2026) is the first RTL GPU fuzzer: SIMT-aware generation, trace-driven differential testing against ISA simulators on Verilator models of Vortex and Ventus, 20 bugs and 10 CVEs. Its evaluation is structural-coverage-only, its campaigns time-boxed, and it carries no functional-coverage model, no checker qualification, and no sign-off discipline—by design. Our methodology is the complementary half, and the flows cross-validate: the JALR LSB deviation we report (R1) was independently found by their S2 (filed against the project’s instruction-set simulator;

their RTL findings X1–X3, at a different pin, are disjoint from ours). Formal memory-consistency verification at RTL is addressed for CPU models by RTLCheck [21] and for GPU memory models in ASPLOS [22]; NVBitFI [20] brought systematic fault injection to GPUs for reliability studies. Concurrently, the Ventus project developed GVM, a UVM-like framework for its own design.

2.5. Checker qualification

Mutation-based qualification—proving checkers can fail—is established practice [19]; we apply the discipline end-to-end to a SIMT lockstep flow, which is what allows our pass rates and coverage to be read at face value.

3. The verification environment

A GPGPU executes programs; it is not driven by them. The environment therefore inverts the usual UVM roles (Fig. 1). Five agents surround the DUT: the *host* and *device-configuration-register* (DCR) agents (active) drive the launch protocol—kernel base address, argument pointers, and thread-count registers—through the platform’s control interface; the *AXI* agent is an active *responder*, a reactive slave backed by a memory model that services the DUT’s fetch and data traffic; a *memory* agent covers the non-AXI configuration of the same model; and a passive *status* agent observes busy/-completion state, the retired-instruction count, and the program counter. A virtual sequencer coordinates launch sequences. Stimulus is therefore a *program* (a compiled ELF), and the UVM test class controls only how it is launched, configured, and perturbed—tests and programs are orthogonal

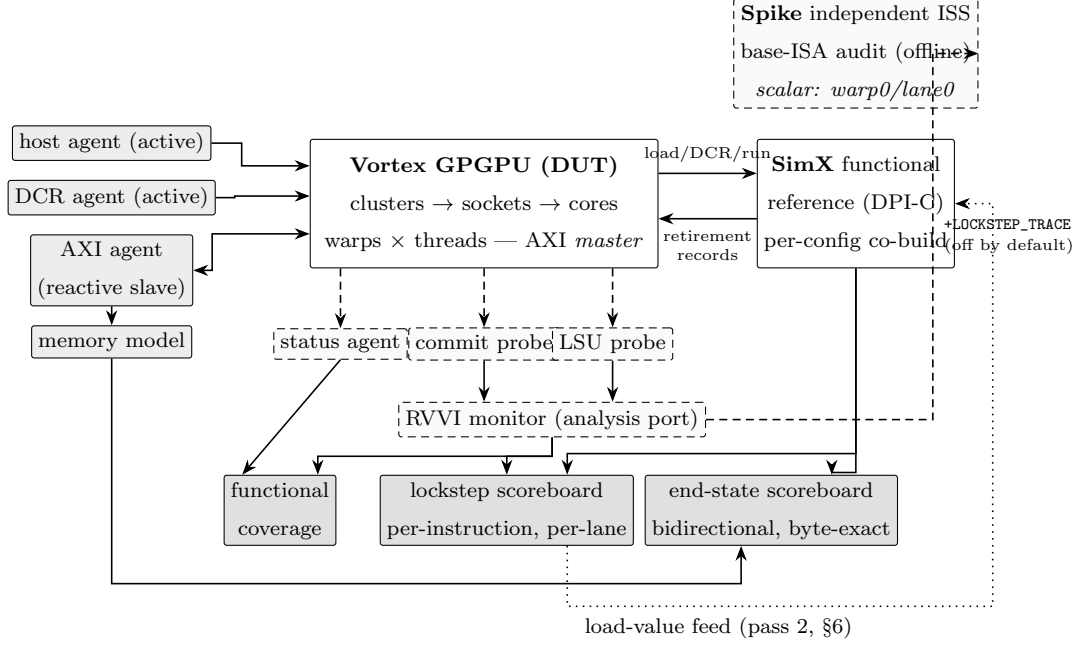


Figure 1: Environment architecture. Solid gray: active UVM agents; dashed: passive probes and offline tools; dark: verdict-rendering scoreboards and coverage. SimX is the *primary* golden model (live over DPI-C, models SIMT, not independent); Spike is a *secondary, independent* scalar ISS (offline, base-ISA subset only). Neither substitutes for the other.

and composed at the build level. The launch space itself is covered systematically: host sequence variants sweep argument-passing conventions, DCR orderings, and poll-vs-interrupt completion styles, driven by a host-coverage test whose covergroups enumerate the launch-space corners.

3.1. Golden-model integration

SimX is linked into the simulation via DPI-C (`simx_init/load/dcr_write/run` plus a per-retirement record export). Both the RTL and SimX are rebuilt per configuration so that DUT and reference always agree on topology. Golden-

model crashes are trapped by a signal handler and mapped to a sentinel exit code, so a reference-model failure is *classified* (Section 4) rather than crashing the run or silently passing it.

3.2. *Passive observability layer*

All white-box visibility is bound, passive, and never a checker. A *commit probe* is bound to the retire arbiter of every core, capturing each retirement beat $\{uuid, wid, PC, rd, wb, tmask, per\text{-}lane\ data\}$ across all clusters, sockets, cores, and issue lanes; an *LSU writeback probe* captures true per-lane load values after sign/zero extension (required because load data is not observable at the commit arbiter, Section 10); an RVVI-style interface and UVM monitor publish the merged retirement stream through an analysis port; and microarchitectural probes feed the SIMT coverage layer (scheduler, divergence/reconvergence, local-memory bank, memory coalescer, cache-event, register-hazard, and commit-beat probes, Section 8). Lockstep capture is plusarg-gated; with the gate off, runs are proven byte-identical to the plain environment.

3.3. *Checkers, configurability, protocol assertions*

Two independent checkers render verdicts: (1) a **bidirectional end-state scoreboard**—after completion, every word the DUT wrote is compared byte-exact against SimX (forward pass, up to 196,868 words in one cache-tier test) *and* every word SimX wrote that the DUT never wrote is checked (reverse pass, catching dropped stores), with per-byte validity masking for sub-word stores, tolerance-comparison of IEEE-rounding-legitimate FP differences (bounded), and read-only/GOT region exclusion; (2) the

lockstep scoreboard of Section 5. A third, orthogonal gate makes the run verdict *assertion-aware*: any RTL runtime assertion firing fails the run with a distinct exit code, and failing runs are excluded from coverage merging; a dedicated negative test (a deliberate misaligned load whose value is discarded) keeps this gate honest by failing through the assertion path alone while the scoreboards pass. One compiled environment runs any topology from plusargs (+CLUSTERS/+CORES/+WARPS/+THREADS); interface widths are derived from the RTL package, and elaboration-time assertions compare every UVM parameter against its DUT counterpart, failing loud on mismatch. Validated topologies include 1CL/1C/4W/4T, 1CL/2C, 2CL/2C/4W/4T, 4C/2W, and 8C/8W/2T. An AXI4 SVA layer (~15–18 protocol properties plus 11 handshake-stability assertions) checks burst legality, outstanding-count consistency, and signal stability under backpressure inline on the DUT’s AXI interface; slave-side *throttle* (ready wait-states) and *flood* (back-to-back read beats) stress modes—plusarg-gated and proven byte-identical when off—exercise the stability properties; and an error-injection mode (+AXI_INJECT_ERR) drives SLVERR/DECERR responses (finding R9, Section 10).

4. Verdicts and non-vacuity

Every run renders one of four verdicts: PASS, FAIL, UNVERIFIABLE, or (for lockstep runs) END-STATE-VERIFIED with an instruction-granularity residual. UNVERIFIABLE is first-class: a run where the golden model cannot render a verdict (for example a reference-model abort) is classified with root-cause evidence—never force-compared, never silently dropped, and never counted as a pass.

Verdicts are only as good as their ability to go red. The environment therefore carries *permanent fault-injection guards* as regression tests, one per checker: **wrong-value injection** (+INJECT_FAULT) flips one bit of one DUT store and must be caught by the end-state scoreboard; **dropped-store injection** (+DROP_STORE) suppresses one DUT store and must be caught by the reverse scoreboard pass (both guards historically detect at the exact address 0x800075d8); **lockstep injection** (+LOCKSTEP_INJECT) flips one bit of one retirement and must be caught at the exact uuid/PC/lane; and a **register-abstraction corruption** (+DCR_RAL_INJECT) must be caught by the register checker. All guards must stay red on injection after any checker change—the mutation-qualification discipline [19], applied end-to-end to a SIMT lockstep flow. During the work, a nearly-accepted *vacuous* gate pass (negative tests reporting PASS when their injection plusargs were absent) was caught and codified into the rule: *always verify the injection message, never the verdict*. This discipline, rather than any single checker, is what allows the coverage and pass-rate numbers of Section 11 to be taken at face value.

5. Per-instruction lockstep for SIMT: five rules, semi-formalized

5.1. The alignment problem

Let D be the DUT’s record stream per (core, warp), each record d carrying five fields (identifier, active-lane mask, PC, destination register, per-lane values), and G the reference’s instruction stream g over the same fields, in program order. Comparing D and G naively produces false mismatches from five structural divergences. Each rule below is stated as *precondition* $\Rightarrow$ *soundness property*: if the precondition holds for the DUT, the transforma-

tion cannot mask a genuine defect (it may only remove false positives), and the residual comparator is exact outside the declared exception classes.

5.2. Rule 1 — retire order (sort by issue counter)

Precondition: the per-(core, warp) issue counter is strictly monotone in program order and does not wrap within a run. *Property:* sorting D by `uuid` per (core, warp) yields program order; hence record d_i aligns with instruction g_i by position. A retirement whose (PC, rd) differ from g_i remains visible. (Observed DUT behavior satisfies the precondition: an earlier-issued instruction was observed retiring after two later ones; the arbiter is `VX_commit.sv:56-71`.)

5.3. Rule 2 — cross-key (program-order keying)

Precondition: each stream independently enumerates the same per-(core, warp) instruction sequence. *Property:* aligning by per-(core, warp) ordinal position is well-defined without any shared identifier; the DUT `uuid`'s high bits supply (core, warp) attribution with no RTL change. Model-side identifiers (SimX leaves `uuid` at zero) are never used as keys.

5.4. Rule 3 — split retirements (mask-union aggregation)

Precondition: every record of one architectural instruction shares its `uuid`, and the union of the records' thread masks equals the instruction's active-lane set L (records may overlap; observed `0xd` then `0xe`). *Property:* aggregating records by `uuid` with mask union, then comparing per active lane, is equivalent to comparing the instruction itself: every lane in L is compared exactly once semantically, and no lane outside L contributes.

5.5. Rule 4 — load data (own tap, region-filtered)

Precondition: (i) a tap where true post-extension load values are observable (the commit arbiter’s data field is stale for loads); (ii) a decidable predicate $V(a, v)$ that is true only when the two models’ memory states agree at address a (initialized region, no preceding divergent write, reference value not initialization poison). *Property:* comparing only lanes with V is a sound *under-approximation*: it never masks a defect (non- V loads are deferred to the byte-exact end-state compare) at the cost of comparing fewer sites. (Empirically: 429 false mismatches on a known-good kernel without the filter; 74 compared / 113 filtered / 0 false with it.)

5.6. Rule 5 — volatile CSRs

Precondition: a statically known CSR class whose values are timing-defined rather than architecturally defined (`mcycle/minstret/mhpmcounter*`). *Property:* excluding their *data* (still comparing PC and destination) removes only comparisons with no architectural ground truth; soundness is preserved, completeness is reduced by exactly that class.

5.7. Declared exception classes and the composite claim

Beyond Rules 4–5, the comparator declares one bounded, logged tolerance: the DUT’s `fsqrt.s` deviates by ≤ 1 ULP from correctly-rounded IEEE 754 [14], tolerated only for `fsqrt` (same-sign, finite, ≤ 1 ULP via `fp32_within_ulp`), tallied in `n_fp_ulp_tol`, with a two-pass reconvergence feed re-checking all downstream operations bit-exactly. The composite soundness claim: *if the preconditions of Rules 1–3 hold and both models faithfully*

Table 1: Lockstep configuration matrix (all lane-exact: 0 mismatches, 0 orphans).

Configuration	Matched writebacks
1CL/1C/4W/4T (vector-add)	1,035 / 1,035
1CL/1C/4W/4T (nested divergence)	2,668 / 2,668
1CL/2C/4W/4T	1,801 / 1,801
2CL/2C/4W/4T	3,333 / 3,333
1CL/1C/2W/2T	855 / 855
1CL/1C/8W/4T	1,423 / 1,423

execute the same program, then any retirement whose (PC, destination, per-lane value) differs from the reference on an active lane, outside the declared exception classes, is reported as a mismatch. The claim is conditional—the preconditions are DUT obligations an adopter must re-derive, listed as such in Section 12.

5.8. Validation

The lockstep was validated lane-exact—zero mismatches, zero orphans—at five configurations (six programs) spanning a $2 \times 2 \times 3 \times 2$ axis space (Table 1), including nested-divergence towers ($3v1 \rightarrow 2v1 \rightarrow 1v1$) that exercised the mask-union aggregation of Rule 3 through divergence and reconvergence with no comparator changes. Tallies are recorded in the verification documentation; per-run raw logs were not retained, and the 8W/4T point predates the reset-relay and parameter-audit fixes, documenting the comparator mechanics rather than current-design status.

6. Verifying racy programs: the two-pass load-value feed

Fenceless multi-core programs have no architecturally defined result on a weakly coherent machine, so per-instruction comparison diverges legitimately: the two models resolve the same race differently. Rather than waive such programs, we make them verifiable. Pass 1 records the DUT’s per-lane loaded values at divergent in-region load sites, keyed by (core, warp, PC, occurrence); pass 2 re-runs the reference following those values, so the models share the race outcome and the *residual*, not the feed, is the verdict: a difference not explained by a fed load stays a hard error. This is a load-value feed in the RVVI sense [6], and it is not suppression.

Soundness assumption, stated and (partially) discharged. The method rests on an assumption we state rather than discharge in general: *that a divergent load is a genuine race resolution, not a DUT load-data defect.* For the flagship case the assumption was confirmed by trace analysis: the first-divergence load reads a word still holding its pristine ELF initialization value (nothing had overwritten it before the DUT’s read), in a single-hart program running on four shared-memory cores—a genuine cross-core ordering race. There, 20 racy loads cascading into 138 divergent retirements collapse to a residual of **zero over 5,432 retirements** (5,314 matched, 118 data mismatches at pass one; 5,432/5,432 matched at pass two), upgrading the verdict from UNVERIFIABLE to *verified modulo the fed race resolutions*.

The boundary, characterized rather than waived. Two cases sit outside the fixed point. First, asynchronous interrupts: an interrupt test collapses 116 cascaded divergences to 7 but not to 0, because the timing-accurate DUT and

the functional model take the interrupt at different instruction boundaries—and re-keying the feed from a per-warp ordinal to $(cid, wid, PC, occurrence)$ leaves the residual identical, proving the cause is genuine and not an alignment artifact. End-state comparison still passes, so the disposition is: end-state VERIFIED, instruction-granularity residual classified. Second, control-flow-steering races: when racy loaded bytes steer branches, pass-2 replay walks a different path and meets *fresh* racy loads the pass-1 trace never keyed (residual 15, thirteen of them load-class; the post-feed end-state compare is clean)—two-pass replay has no fixed point when races feed branches, and the verdict is left honestly red. The generalization: *two-pass trace replay is a fixed point for data-only divergence; asynchronous-input timing and race-steered control flow require a step-follower reference.*

7. Stimulus

Directed kernels. (~ 30 , all compiled with the Vortex toolchain, all byte-exact against the reference): arithmetic and memory baselines; all 13 FPU operation classes; tensor-unit WMMA including multi-warp collective launches; nested-divergence towers (asymmetric $3v1 \rightarrow 2v1 \rightarrow 1v1$ splits); barriers under partial thread masks; **wspawn**/**tmc** sweeps; cache miss-status-holding-register and memory-pressure stress; a 232 KB-text kernel; a 256 KB high-entropy store stress; division corner cases; CSR writes under SIMT masks; and VOTE/SHFL operations.

Constrained-random. A riscv-dv [7] pipeline targets rv32im with GPU-specific post-processing (machine-mode CSR stripping, **ecall** $\rightarrow$ **ebreak**) and end-state verification. Twelve profiles were brought up; two are excluded as *unim-*

plementable on this DUT—unaligned-load/store (no misaligned support) and illegal-instruction (no trap architecture)—and one was later dropped as a byte-identical duplicate of another profile, leaving nine generator profiles (twelve seeded instances run in the suite). Jump-stream generation is sanitized to keep the generator’s deliberate JALR-LSB probing away from the R1 deviation it would otherwise trip; the deviation itself is reported (Section 10), not hidden.

SIMT-aware generation (simtgen). An original random C-kernel generator targets the microarchitectural axes scalar generators cannot reach, with knobs derived from the DUT’s own configuration space (warp/thread counts, LSU coalescer window, local-memory bank count, IPDOM stack depth), informed by the generation-axis taxonomy published by FuzzGPU [3] but implemented against our own checking stack; no third-party code is used. `simtgen` implements four axes: divergence (structured branch trees with controlled split depths and asymmetric masks), memory (stride/scatter/bank-hostile access patterns), barrier topology (single-warp barriers, deadlock-safe by construction; `wspawn` is deliberately never emitted—it is a runtime-only bootstrap primitive with its own structural waiver), and VOTE/SHFL (all eight intrinsics per program). The barrier and VOTE/SHFL axes closed no new coverage—their targets were already fully covered by dedicated directed kernels; their contribution is generator completeness and seed-diversity robustness evidence. A ten-seed sweep across all nine riscv-dv profiles (90 additional distinct programs, content-hash verified, all passing) produced *no measurable functional-coverage gain* (every category bit-identical except toggle at +0.06%)—a robustness result reported as such, not as coverage progress.

Protocol stress. The AXI throttle, flood, and error-injection modes of Section 3, plus host/DCR launch-space sweeps.

8. A three-layer coverage model

The functional model is organized in three layers with different owners and deliberately different blind spots, and their databases are never merged:

L1 — ISA coverage (third-party riscvISACOV): 80 active covergroups over RV32I/F/M/Zicsr/Zifencei. Sampling *every active lane as a hart* (4,581 samples) was proven to cover exactly the same bin set as lane-0-only sampling (1,677) on our kernels—an empirical demonstration that ISA coverage is blind to the SIMT axis, which is why L2 exists. A closed gap-hunt leaves 78/80 covergroups real and 429/516 target bins hit (83.14%; 89.28% weighted) after register-index exclusions; two covergroups are permanently empty by construction.

L2 — SIMT microarchitecture (this work’s contribution): probe-sampled covergroups for warp divergence crossed with IPDOM stack depth (`divergence_cg`), reconvergence, barrier scope and events, `tmc/wspawn` behavior, scratchpad bank conflicts (`lmem_bank_cg`), memory-coalescing classes (`coalesce_cg`), cache events, register hazards, and commit-beat shapes (`beat_cg`), alongside instruction-class groups per execution unit. The closures of this layer, their attribution corrections, and its one coverage-model defect are reported in Sections 11 and 9.

L3 — system and protocol (ours): SVA assertion and directive coverage, AXI transaction space, DCR/host launch spaces, and system-state crosses.

9. Exclusion methodology: machine-generated, gate-checked waivers

Closure claims live and die by what they exclude. This section elevates the exclusion machinery—largely a subsection in the conference treatment—into a named, reusable methodology with three rules and one gate.

Rule 1: exclusions are structural, RTL-cited, and machine-generated. A per-configuration generator emits every waiver with a `file:line` citation—for example, a write-response stability assertion is unreachable because the adapter hardwires `m_axi_bready = 1'b1` (`VX_axi_adapter.sv:313`), and the global-barrier path is excluded only at single-core, where it is structurally unreachable, while remaining live at ≥ 2 cores. Waivers are config-aware in both directions: RV64-only behavior (LD/SD; float-double conversion) is excluded at RV32 by `ignore_bins` keyed on XLEN, and the passthrough L2/L3 storage (`VX_cache_wrap.sv:160`, built only when `PASSTHRU==0`) was excluded by the EUR class *until* the Phase H campaign replaced that waiver with genuinely exercised coverage (Section 11).

Rule 2: reachable-but-unhit stays red. Bins that stimulus could reach but did not are reported uncovered, never waived. The adversarial check is deliberate: when `mshr_flood` drove 67,207 dcache misses and its target bin still did not hit, the outcome was recorded as a finding (OBS-031), not a waiver.

Rule 3: ceilings are root-caused, not waived. The toggle plateau was traced per sub-module to a read-only icache data port (51,340 bins, 55.7% sub-module toggle; 22,730 bins missing—26.4% of the entire toggle gap—traceable to `VX_socket.sv:106` tying `.WRITE_ENABLE(0)`) and to constant high address bits; an adversarial maximum-entropy kernel moved aggregate toggle

by +0.02%, establishing the ceiling as structural rather than a stimulus short-fall.

The gate. A blocking *hits-invariant* check runs at every coverage merge: each waived bin must have had *zero* hits in the unwaived bank, and exclusions may shrink the denominator only—any change to covered-bin counts aborts the merge. In operation this gate has caught three real defects of three different classes: two waiver defects early on (one latent for weeks), a stale-staging contamination of an isolated merge, and—during this campaign—an exclusion set generated with 1CL configuration keys against a 2CL merge (a 460-bin toggle discrepancy), correctly aborting the merge. A methodology whose own bookkeeping tooling catches its operator’s errors is, we argue, the minimum bar for trusting any signed-off percentage; the gate itself is simulator-idiomatic but simulator-independent in concept, and is reusable wherever merged coverage databases drive sign-off.

Coverage definitions are themselves verification targets. The bank-conflict coverpoint was originally defined over *accepted* requests; since the local-memory crossbar admits exactly one winner per bank per cycle, “two accepted requests on one bank in one cycle” was structurally impossible—the bin could never hit, and deliberate bank-hostile stimulus proved it by producing zero conflicts. Reclassifying the coverpoint onto *offered* requests made the bin both reachable and honest (169 conflict hits, 61 no-conflict, 70,460 idle). A coverage model must be validated against the RTL’s structure with the same discipline as the RTL itself; the stimulus gap and the taxonomy defect were disentangled by experiment, and the closure credit is shared between the probe fix and the generator.

10. Findings

Every finding is maintained in an evidence-cited register shipped with the work: one entry per finding—what was seen, `file:line` and/or run evidence, classification, and disposition. Table 2 summarizes the RTL findings R1–R10; the two developed below are the ones whose stories carry methodological weight.

10.1. RTL defects and hazards

An unknown reset for one cycle, found by restoring a silenced assertion (R10). The design distributes reset through a relay that registers the incoming reset in a flip-flop which has no initial value and which nothing resets:

```
'PRESERVE_NET reg [R-1:0] reset_r;    // no initial value
always @(posedge clk) reset_r[i] <= reset;
assign reset_o[i] = reset_r[i / F]; // X until first edge
```

Every module behind a relay therefore observes an *unknown reset for one cycle*: `if (X)` is not true, so reset-held logic takes its non-reset branch and executes, and assertions inside those branches evaluate on unknown operands. The path to the finding is the point: library counter assertions fired during bring-up and were guarded off (skip on unknown inputs) so the environment could run—for months. Restoring the original assertion and root-causing it exposed the relay; we replaced it with an `async-assert/sync-deassert` synchronizer, after which the counter assertions pass with no guard (twelve firings become zero), a 51-run regression passes with assertions armed, the injection guards were re-confirmed, and the local modification was retired in favor of the unmodified upstream file. The defect remains upstream. Two observations generalize: *a checker disabled to make a bench run is a checker whose*

findings are lost—silently and open-endedly; and fixing the defect reduced branch coverage (95.1%→94.5%, indicative) because modules behind a relay had been executing normal-operation paths during an illegitimate state and those executions had been counted as covered branches—no coverage metric can report that some of its coverage came from a state that cannot legitimately arise.

JALR does not clear the target LSB (R1). The RISC-V specification [13] requires JALR to clear the target LSB; Vortex omits the clear (`VX_alu_int.sv:222`), and with no trap architecture cannot raise the prescribed exception for a misaligned target. In the debug build the odd bit survives as the *architectural* PC: fetch silently word-aligns so execution continues on correct instruction words, but `auipc/la` inherit the skew, link registers accumulate it across chained jumps (observed $+1 \rightarrow +3$), and downstream accesses go misaligned, cascading into R3. The trigger is spec-legal stimulus—`riscv-dv` deliberately sets the JALR base LSB expecting the architectural clear—so roughly half of generated jumps derail; every profile of a twelve-profile suite fired misaligned-access assertions (30–7,616 per run). FuzzGPU found the same deviation independently in the project’s instruction-set simulator (S2, upstream PR #339, CWE-682), and their RTL findings X1–X2 were in turn exercised at our pin (Section 11): X1 reproduces, and X2’s divergence is the *reference model’s* fault—the symmetry of that exchange is the cross-validation argument in its purest form.

Further verified defects. `fsqrt.s` is one ULP off correctly-rounded IEEE 754 (lockstep lane-0 mismatch localized to the sqrt unit; the end-state tolerance absorbs it, and only per-instruction comparison surfaces it—see Section 5

for the full exception-class handling); the memory scheduler’s timeout is denominated in simulation-time picoseconds, flooding L2/L3-enabled runs with 22,187 false alarms while functionally clean; and the write path is fire-and-forget with *no AXI error path*—injecting SLVERR/DECERR every seventh transaction produced exactly 166/166 assertion firings and no recovery path.

10.2. Reference-model defects

A central lesson of this work: *a lockstep flow verifies the reference model as much as the DUT*. SimX read instructions at the exact byte PC; a computed jump to an odd target (tolerated by the RTL, which word-aligns) made the reference read across an instruction boundary and abort, silently classifying runs unverifiable. The lockstep’s own divergence report localized the defect; word-aligning the reference fetch recovered the runs. The *obvious* alternative—masking the JALR target in the reference—was tried and *reverted with evidence*: it produced 21,968 phantom PC mismatches, because it fixed the model to the specification rather than to the DUT. Beyond it: SimX aborts on unknown input at 56 disassembly-verified sites (35 decode, 21 execute), mostly in the pretty-printer—a run can be unverifiable because the golden model could not *name* an instruction, which the lockstep upgrades to “byte-exact for all 17,664 retirements before the abort”; and a SIMT-divergent CSR access produced 109 genuine mismatches (the reference serialized lanes over the shared per-warp `fcsr` while the RTL reads-broadcast and writes lane 0), corrected in the reference and documented in the RTL as a visible architectural choice. The X2 reproduction (Section 11) extends the lesson to externally-sourced test cases: a fuzzing campaign’s “RTL bug” was, at our pin, a golden-model bug that only a DUT-vs-reference compari-

son could attribute correctly.

11. Results

11.1. Closure campaign and bank provenance (W1)

The campaign concluded with a deliberate scope decision rather than a re-run for documentation-literalism: the divergence and memory closures are *in the suite bank* (`bank_1CL_1C_4W_4T_L2_20260909`: 23 covergroups, 504/524 bins = 96.18%, total 94.55% on the grown post-remediation coverage model); the barrier and VOTE/SHFL generator axes, built and verified by isolated merge (`simtgen_barrier_vote_shfl_20260910`), closed zero new bins because their targets (`cp_vote_shfl_op` 8/8, `cross_sfu_threads` 33/33) were already fully covered by dedicated directed kernels—an attribution correction to the initial reporting that we state rather than quietly keep. A relay-fixed 2CL bank (`bank_2CL_2C_4W_4T_relayfix_20260910`: 110 runs, 0 failures, 93.39% bins / 94.29% total) restores cross-configuration comparability on the fixed design. A third configuration point then closes the coverage story’s weakest waiver: shared L2/L3 had only ever run *passthrough-waived* (the EUR exclusion class)—a waiver standing in for untested RTL. A seven-kernel cache-focused suite at 2CL with L2/L3 enabled on the relay-fixed design (all runs passing; the `cache_tier` phase is the project’s longest single run at ~2 h45 m) produced a dedicated bank whose liveness was verified recursively—real `cp_hit` counts on every new cache covergroup instance (L2: 86,503 and 88,104; L3: 36,501 and 81,854)—and, with the passthru waivers correctly not generated, a correspondingly enlarged denominator: 85.10% total (covergroup bins 1,087/1,680 = 64.70%). Expectations are set honestly:

this is coverage and configurability evidence, not bug-finding—shared caches change timing, not architectural outcome, and the functional validation of L2/L3 predates it (57,379 matched lockstep pairs, 2026-08-07). With this bank, the coverage-run checklist that governed the campaign is fully closed, recorded in a final merged sign-off report. During this campaign the blocking hits-invariant waiver gate caught a real operator error—an exclusion set generated with 1CL keys against a 2CL merge—and aborted the merge; this is the third class of defect the gate has caught in operation, recorded here as evidence for the exclusion methodology of Section 9.

11.2. Verification cost (W2)

Lockstep’s wall-clock overhead is within run-to-run noise at every program size (± 1.5 s on ~ 15 – 21 s baselines; ± 20 s—inside the inter-rep spread of plain mode—on the ~ 860 s kernel). Peak memory shows one real step: the second (SimX) model instance adds $\sim 29\%$ RSS at the large working set and $\sim 1.5\%$ at the small ones; the load-value feed adds no further measurable memory. The one FAIL in the grid is itself informative: `fpu\_test` under plain lockstep (feed disarmed) reproduces the known fsqrt 1-ULP deviation cascading into downstream mismatches—exactly the behavior Section 5’s exception-class design predicts—while the same program under lockstep+feed passes with residual zero: the reconvergence feed, not the tolerance alone, absorbs the cascade. A measurement artifact is disclosed rather than hidden: an initial `wide_stress` run used an undersized timeout budget, produced false TIMEOUT verdicts on all six rows, and was discarded (budget artifact, OBS-042 precedent); all reported rows completed with `rc=0`.

11.3. FuzzGPU PoC reproduction at our pin (W3-B)

FuzzGPU’s three Vortex RTL findings (filed at a different commit) were each exercised at our pin under this environment:

- **X1 (FPU compare-to-x0 unconditional writeback, PR #356): reproduced.** The PoC fires the RTL’s own `invalid writeback register` runtime assertion at the PoC’s exact `feq.s x0` PC; the defect is present and unfixed at our pin.
- **X2 (reserved M-extension funct7 decode, PR #358): reproduced—and re-scoped.** The upstream fix lives entirely in the reference model’s decoder; our RTL was always exact-match and never had the defect. A hand-encoded reserved opcode produces a genuine DUT-vs-SimX divergence in which the *hardware is right and the golden model is wrong* (DUT decodes ADD; SimX decodes MUL)—the paper’s central thesis (the reference model must be verified alongside the DUT) confirmed by an externally-sourced test case, and a caution against citing X2 as an RTL bug.
- **X3: static-only** (engineering cost of dynamic reproduction outweighed the value; documented, not attempted silently).

These runs are deliberately not suite members: X2’s expected outcome is a real mismatch, so it is kept as a directed qualification case instead.

11.4. Marginal coverage per generator kind (W3-A)

The ordering—directed > simtgen > riscv-dv—quantifies, on one axis, the claim the seed farm first drew for riscv-dv alone: the SIMT stimulus gap

needed a SIMT-aware generator, not more scalar seeds; and even a SIMT-aware generator cannot replace coverage-model validation (the bank-conflict and coalescing attribution lessons of Section 8).

11.5. Bug-discovery dynamics (W7)

Figure 2 plots cumulative register findings against project time, annotated with the methodology milestones that unlocked each cluster: the first lock-step bring-ups (reference-model defects became visible), the assertion-aware verdict gate (R10 surfaced the moment its silenced assertion was restored), the two-pass feed (racy-program findings), and the waiver gate (coverage-model defects). The curve is the paper’s process argument in one picture: checking depth, added in stages, kept producing findings on both sides of the comparison for the entire project—there was no point at which the methodology saturated into vacuous passes.

12. Threats to validity

12.1. Internal validity

The two-pass feed’s soundness rests on the assumption that a divergent in-region load is a genuine race resolution; we state it, discharge it with trace-level witnesses where possible, and classify rather than force-green elsewhere. Differential testing is structurally blind to stimulus faults (shared-binary blindness); one input class is closed by independent assertions, the class is reported open. Comparator tolerances are enumerated (load filter, volatile CSRs, fsqrt 1-ULP) and logged, never silent. Two infrastructure gaps are recorded rather than assumed away: no in-sim reset-testing methodology

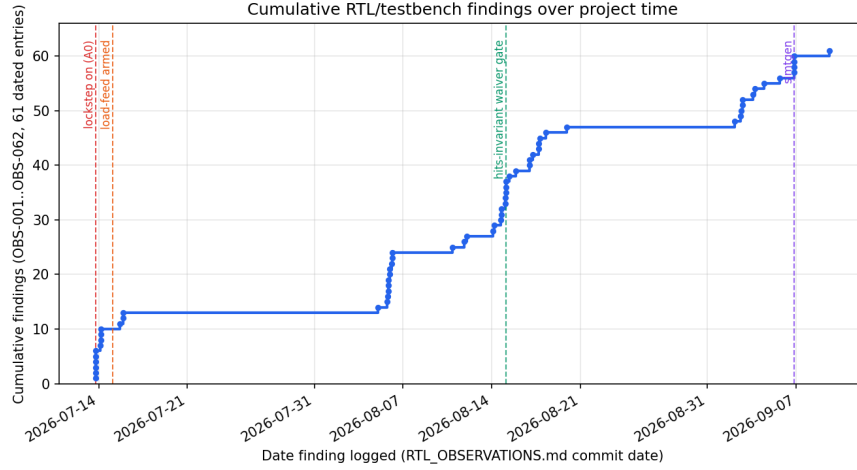


Figure 2: Cumulative findings (OBS register) over project time with methodology-milestone annotations. Data and reading: `figures/bug\_discovery\_curve.csv` / `.md` in the artifact.

exists (power-on reset only—the R10 bug class has never been exercised under randomized reset), and the environment has no isolated component-level test layer; both are open items, not silently covered claims.

12.2. External validity

All results are one DUT at one pin with locally modified files (disclosed), one simulator, one build; the alignment rules’ DUT preconditions are stated so adopters can re-derive them; transfer beyond is belief, not demonstration. Open-source UVM simulation is maturing (upstream UVM 2017 elaborates in Verilator with constrained randomization), but SystemVerilog functional coverage remains unsupported there, so the coverage-driven half requires a commercial simulator today.

12.3. Conclusion validity

Coverage figures are per-configuration and never blended; frozen banks are never overwritten; the D-extension elaboration is scoped by inspection *and* confirmed by an EXT_D-off recompile with a coverpoint-by-coverpoint bin diff (zero scored bins change; the entire delta is confined to two weight-0 coverpoints already excluded from every reported percentage); per-mnemonic claims avoid aliased encodings; bank provenance and design state accompany every table.

13. Toward silicon sign-off

The work reported here is *front-end functional verification* of an RTL model. Silicon sign-off requires substantially more, and the two categories below are deliberately separated because they demand different tooling, different skills, and—critically—different claims. Neither list is a statement of partial progress: every item is currently absent unless noted.

Remaining front-end (RTL functional) work. (1) Stimulus diversity, not volume—the seed-farm negative result shows coverage saturating inside the scalar generator’s reach; the remaining work is stimulus of a different *kind*, partly delivered by the error-injection mode and the completed generator axes. (2) A reset-testing methodology—none exists in simulation (power-on reset only); R10 proves the bug class is real, and several unreset elements have been identified structurally but never exercised under randomized reset. (3) Formal property verification on arbitration-heavy control (cache miss-status handling, commit arbitration), where dynamic simulation is weakest and the state space is small enough for proof; the IPDOM reconver-

gence stack (push/pop balance, no underflow, reconvergence restores the entry mask) is the natural first target. (4) An isolated component-level test layer (none exists). (5) Coverage-model provenance: the L2/L3 layers are self-authored rather than traced to a specification document, so closure measures the model, not the specification. (6) An independent SIMT reference—structurally absent, as stated in the threats section.

Back-end and ASIC sign-off (entirely out of scope). Gate-level simulation with back-annotated timing; static timing analysis across PVT corners; DFT (scan, ATPG, MBIST); CDC/RDC analysis with metastability modeling; lint and structural sign-off; low-power verification; physical-design closure; equivalence checking between RTL, netlist, and post-layout; and a post-silicon path back into this regression. A useful way to read the two lists: the front-end items would raise the strength of the claims made in this paper, whereas the back-end items are prerequisites for a different claim entirely—that the design is manufacturable and will function in silicon. Nothing in this work speaks to the latter.

14. Conclusion

This paper presented a complete UVM 1.2 environment and a functional-coverage closure methodology for an open-source RISC-V GPGPU, and reported what the resulting checking depth revealed on both sides of the comparison. Five SIMT alignment rules—each semi-formalized as a precondition—soundness property, each with its DUT obligations stated—make per-instruction, per-lane lockstep sound for a warp machine; a two-pass load-value feed verifies racy fenceless programs at instruction granularity (residual zero over

5,432 retirements, verified modulo fed race resolutions) with its boundary characterized rather than waived; a three-layer coverage model—including, to our knowledge, the first published SIMT functional-coverage layer for RTL GPU verification—was closed under machine-generated, RTL-cited exclusions enforced by a blocking waiver-integrity gate that caught three classes of defect in operation, including its own operator’s; and every verdict is backed by injection-proven checkers. The checking depth paid for itself in findings: an ISA deviation independently corroborated by concurrent fuzzing work (and reciprocated—their findings, exercised at our pin, split into a real RTL defect and a golden-model bug), a reset-relay X window found by unsuppressing an assertion, and a reference-model fetch bug found by the lockstep itself. The central lesson—*the reference model must be verified alongside the DUT, the coverage model alongside both, and the checkers above all three*—transfers to other open SIMT designs, and fills the half of the open-GPU verification problem that fuzzing does not address.

Acknowledgment

The authors thank the Vortex group at Georgia Tech and the maintainers of the open verification assets (core-v-verif, riscv-dv, RVVI, riscvISACOV, Spike) leveraged by the environment.

References

- [1] B. Tine, K. P. Yalamarthi, F. Elsabbagh, H. Kim, Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics, in: Proc. 54th

- IEEE/ACM Int. Symp. Microarchitecture (MICRO-54), 2021, pp. 754–766.
- [2] F. Elsabbagh et al., Vortex: OpenCL compatible RISC-V GPGPU, in: Workshop on Computer Architecture Research with RISC-V (CARRV), 2019.
- [3] Z. Gao, J. Wang, Q. Zhou, L. Shan, J. Hu, X. Jia, Z. Lv, Fuzzing open-source GPU hardware with SIMT program generation, in: Proc. 35th USENIX Security Symposium, 2026, pp. 2465–2484.
- [4] Y. Xu et al., Towards developing high-performance RISC-V processors using agile methodology, in: Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO-55), 2022.
- [5] OpenHW Group, core-v-verif, <https://github.com/openhwgroup/core-v-verif>.
- [6] OpenHW Group / Imperas, RVVI: RISC-V Verification Interface, <https://github.com/riscv-verification/RVVI>.
- [7] CHIPS Alliance, riscv-dv, <https://github.com/chipsalliance/riscv-dv>.
- [8] RISC-V International, Spike, the RISC-V ISA simulator, <https://github.com/riscv-software-src/riscv-isa-sim>.
- [9] Imperas Software, ImperasDV, <https://imperas.com/imperasdv>.
- [10] lowRISC, OpenTitan, <https://github.com/lowRISC/opentitan>.

- [11] IEEE Standard for Universal Verification Methodology Language Reference Manual, IEEE Std 1800.2-2020, 2020.
- [12] IEEE Standard for SystemVerilog, IEEE Std 1800-2017, 2017.
- [13] A. Waterman, K. Asanović (Eds.), The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, 2019.
- [14] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, 2019.
- [15] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, P. Thomson, The design and implementation of a verification technique for GPU kernels, *ACM Trans. Program. Lang. Syst.* 37 (3) (2015) 10:1–10:49.
- [16] G. Li, P. Gopalakrishnan, I. Ghosh, S. P. Rajan, G. Gopalakrishnan, GKLEE: Concolic verification and test generation for GPUs, in: *Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2012, pp. 215–224.
- [17] T. L. Sharma, M. Bauer, A. Aiken, Verification of producer-consumer synchronization in GPU programs, in: *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2015, pp. 88–98.
- [18] C. Cummins, P. Petoumenos, H. Leather, S. Fursin, Compiler fuzzing through deep learning, in: *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2018, pp. 95–105.
- [19] L. Di Guglielmo, F. Fummi, G. Pravadelli, Vacuity analysis for property

- qualification by mutation of checkers, in: Proc. Design, Automation & Test in Europe (DATE), 2010, pp. 262–267.
- [20] T.-T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, M. B. Glick, S. W. Keckler, NVBitFI: Dynamic fault injection methodology for GPU applications, in: Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN), 2021, pp. 64–76.
- [21] Y. A. Manerkar, D. Lustig, M. Martonosi, M. Pellauer, RTLCheck: A property checking approach for custom memory consistency models, in: Proc. 50th IEEE/ACM Int. Symp. Microarchitecture (MICRO-50), 2017, pp. 713–725.
- [22] D. Lustig, S. Sahasrabuddhe, M. Giroux, A formal analysis of the NVIDIA PTX memory model, in: Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019, pp. 253–266.
- [23] Siemens EDA, QuestaSim, <https://eda.sw.siemens.com/en-US/ic/questa/simulation/>.
- [24] Verilator open-source project, <https://www.verilator.org>.
- [25] Verilator project, UVM base libraries with edits for Verilator, <https://github.com/verilator/uvm>.
- [26] PlanV, Enabling UVM support in Verilator (blog series), <https://planv.tech/blog/>.

Table 2: RTL findings. The R1 spec deviation was independently found by FuzzGPU [3] in the ISS component (S2, PR #339); their RTL findings (X1–X3) were exercised at our pin (Section 11); R2 was independently fixed upstream.

ID	Finding	Class	Disposition
R1	JALR LSB not cleared; odd PC propagates via AUIPC	Bug (ISA)	needs RTL fix; stim- ulus sani- tized
R2	STALL_TIMEOUT 1**N≡1; watchdog never scales	Bug (latent)	fixed; independ- ently fixed up- stream
R3	Misaligned access: no trap; silently retargeted/- torn	Hazard	per SW contract; gated
R4	Core self-starts; DCRs unreset	Hazard	worked around
R5	Per-warp out-of-order commit	Quirk	handled (§5)
R6	One load → multi records, overlapping masks	Quirk	handled (§5)
R7	uuid encodes flat core+warp id	Quirk	exploited (§5)
R8	Load data unobservable at commit arbiter	Observability	closed (LSU probe)
R9	Fire-and-forget writes; <i>no</i> <i>AXI error path</i>	Characteriz.	cited; fault- injected (166)
R10	Reset relay: reset regis- tered in an unreset flop; one-cycle X window	Bug (X)	fixed; still upstream

Table 3: Wall-clock and peak-RSS cost of the checking stack (median-of-two reps; QuestaSim 2021.2, 1CL/1C/4W/4T). Four programs spanning the kernel-size spectrum: `vecadd_lite` (~ 9.9 k cycles), `diverge_lite` (divergence kernel), `fpu_test` (FPU), `wide_stress` (256 KB high-toggle).

Program	Mode	Wall clock	Peak RSS
<code>vecadd_lite</code>	plain	15.3 s	296 MB
<code>vecadd_lite</code>	lockstep	15.0 s	299 MB
<code>vecadd_lite</code>	lockstep+feed	14.8 s	299 MB
<code>diverge_lite</code>	plain	17.8 s	297 MB
<code>diverge_lite</code>	lockstep	18.2 s	301 MB
<code>diverge_lite</code>	lockstep+feed	18.1 s	301 MB
<code>fpu_test</code>	plain	21.0 s	296 MB
<code>fpu_test</code>	lockstep	20.7 s	301 MB [†]
<code>fpu_test</code>	lockstep+feed	21.0 s	301 MB
<code>wide_stress</code>	plain	851 s	549 MB
<code>wide_stress</code>	lockstep	871 s	708 MB
<code>wide_stress</code>	lockstep+feed	856 s	708 MB

Table 4: Marginal covergroup-bin contribution per program kind. Directed kernels are hand-targeted at an identified gap, so their per-program rate is not comparable on the same axis as the two generators; the two generator rows are measured directly.

Generator	Programs	Marginal contribution
riscv-dv (seed farm, seeds 2–11 $\times$ 9 profiles)	90	0 bins (370/377 unchanged; toggle +0.06% only)
simtgen (divergence + memory axes)	57	cp_split_depth 3/4 $\rightarrow$ 4/4; cp_bank_conflict 0/3 $\rightarrow$ 3/3 (credit shared with the OBS-060 probe reclassification)
simtgen (barrier + vote_shfl axes)	6	0 bins (targets already covered by directed kernels vote_shfl, bar_masks, sfu_masks)
Directed kernels (representative)	—	lmem_stress: scratchpad toggle 57.5% $\rightarrow$ 73.2%; multicore_isa: closed the per-core gap across all 4 cores; mshr_flood: 67,207 misses driven, target still unhit — itself finding OBS-031, not a closure